Universidade do Minho
Departamento de Informática
Licenciatura em Ciência de Dados

Sistemas Conexionistas para Aprendizagem Automática
3º Ano, 1º Semestre
Ano letivo 2026/2027

Enunciado Prático nº 3
07 de outubro de 2025

Tema

Introdução ao *PyTorch*: Tensores e *Autograd*.

Enunciado

Pretende-se, com esta ficha, que seja realizado um conjunto de tarefas que permitam uma maior compreensão da API do *PyTorch*. Pretende-se também promover um espírito crítico e de investigação na utilização da diferenciação automática (*autograd*), que está na base do treino de uma rede neuronal.

Tarefas

Esta ficha encontra-se dividida em duas partes distintas.

1. Na primeira parte desta ficha prática pretende-se que sejam resolvidos os seguintes exercícios sobre *tensors*:
   a) Criar dois *tensors* de *rank* 0, a e b, de qualquer valor. Retornar a+b se a>b senão a-b;
   b) Criar dois *tensors* de *rank* 0, a e b, de qualquer valor aleatório entre -1 e 1. Retornar a+b se a<b; a-b se a>b; e 0 como *default*;
   c) Criar um *tensor*, a, com o valor [[1, 2, 0], [3, 0, 2]], e um *tensor* de zeros, b, com o mesmo *shape* de a (*shape*=(2, 3)). Retornar um *tensor* booleano com o valor *True* para cada elemento de a igual a b;
   d) Criar um *tensor* 1d, a, com 20 elementos compreendidos entre 1 e 10. Retornar um *tensor* com os elementos de a cujo valor é superior a 7.
   e) Criar um *tensor* 2d, a, com *shape*=(3, 4), contendo valores inteiros entre 0 e 9. Retornar um *tensor* com todos os elementos pares de a.
2. Na segunda parte deste enunciado pretende-se explorar a diferenciação automática (*autograd*) do *PyTorch*:
   f) Criar um *tensor* de *rank* 0, x, com o valor 2.0 e requires_grad=True. Calcular $y = x^3 + 2x^2 - 5$ e obter o gradiente dy/dx com backward(). Confirmar o resultado derivando à mão;
   g) Repetir o cálculo de y e a chamada a backward() do exercício anterior, sem limpar o gradiente. O que acontece ao valor de x.grad? Porquê? Como se resolve?

h) Ajustar uma reta a um conjunto de dados com descida do gradiente «à mão», i.e., sem recorrer a um otimizador. Devem, para esse efeito, utilizar e completar o seguinte excerto de código (procurar pela *tag* ***TODO*** - 8 no total) e analisar os resultados obtidos. Devem também experimentar variar a *learning rate* e o número de *epochs*, analisando e comparando os resultados obtidos:

```
import torch
import matplotlib.pyplot as plt

#reproducibility
torch.manual_seed(TODO)

#synthetic data: y = 2x + 1 (plus some noise)
x = torch.linspace(-1, 1, 100)
y = 2 * x + 1 + 0.1 * torch.randn(100)

#parameters to learn (both start at zero)
w = torch.tensor(0.0, requires_grad=TODO)
b = torch.tensor(0.0, requires_grad=TODO)

lr = 0.1
losses = []
for epoch in range(100):
    #forward: prediction and mean squared error
    y_pred = TODO
    loss = TODO

    #backward: compute the gradients
    TODO

    #update the parameters (outside the graph) and reset the gradients
    with torch.no_grad():
        w -= TODO
        b -= TODO
    w.grad.zero_()
    b.grad.zero_()

    losses.append(loss.item())

print('w = %.3f, b = %.3f' %(w.item(), b.item()))

#plotting the learning curve
plt.plot(losses)
plt.xlabel('epoch')
plt.ylabel('loss')
plt.show()
```